Other Enumerables

Several other classes in Ruby core or the Ruby standard library respond to each and include Enumerable in a useful way. Here's a quick tour:

- ARGF is the combination of all files passed to a command-line tool. ARGF#each iterates over each line in the concatenated set of files. Not each file, each line in the files one by one.
- CSV implements each to iterate over each parsed row in the CSV file.
- Dir implements each for the files in the directory, passing each filename to the block one by one.
- ENV, the repository for environment variables at runtime, behaves like a hash, and each will pass each successive environment variable name and value as a pair.
- IO, the generic input/output class implements each to go line by line through the item being read. Its subclasses, including File and StringIO behave similarly.
- Net::HTTPHeader contains the values in the header of an HTTP request, and each takes a block for each entry in the headers as a key/value pair.
- Range responds to each as a array would, calling it once for every element inside the range.
- Struct responds to each as a hash would, calling the block once for each attribute in the struct as a key/value pair.

Enumerator

Nearly every Enumerable method that takes a block argument can also be called without a block, in which case the method returns an Enumerator. (There are a few subclasses of Enumerator that you wouldn't create by hand but which implement some specialized logic.) In addition to those Enumerable methods, you can create an enumerator in a few other ways.

Creating Enumerators

Any object can directly be converted to an Enumerator with the to_enum method (aliased as enum_for). The argument to to_enum is a symbol that is the name of a method that converts the object to something enumerable, and then any further arguments that might be passed to the method. You can then treat that enumerator like any other enumerator. For example:

```
x = "a string with lots of words"
enum = x.to_enum(:split)
enum.with_index.map { |word, index| [word.upcase, index] }   # => [["A", 0],
                                                              # .. ["STRING", 1],
                                                              # .. ["WITH", 2],
                                                              # .. ["LOTS", 3],
                                                              # .. ["OF", 4],
                                                              # .. ["WORDS", 5]]
```

You can create an infinite enumerator using the produce method, which takes an optional initial value as an argument, and a block. The block takes an argument, and each time the enumerator is invoked, the previous block return value is passed to the block and a new value is returned. This is most useful when combined with lazy or with some kind of find method. This uses map, select, and take to return the first five even triangular numbers:

```
triangular_numbers = Enumerator.produce([1, 2]) do |number, count|
  [number + count, count + 1]
```